

Translated

From: e-maxx.ru

Paint the edges of the tree

This is a fairly common task. Given a tree G with N vertices. There are two types of queries: the first one is to paint an edge, the second one is to query the number of colored edges between two vertices.

Here we will describe a fairly simple solution (using a [segment tree](#)) that will answer each query in $O(\log N)$ time. The preprocessing step will take $O(N)$ time.

Algorithm

First, we need to find the [LCA](#) to reduce each query of the second kind (i, j) into two queries (l, i) and (l, j) , where l is the LCA of i and j . The answer of the query (i, j) will be the sum of both subqueries. Both these queries have a special structure, the first vertex is an ancestor of the second one. For the rest of the article we will only talk about these special kind of queries.

We will start by describing the **preprocessing** step. Run a depth-first search from the root of the tree and record the Euler tour of this depth-first search (each vertex is added to the list when the search visits it first and every time we return from one of its children). The same technique can be used in the LCA preprocessing.

This list will contain each edge (in the sense that if i and j are the ends of the edge, then there will be a place in the list where i and j are neighbors in the list), and it appear exactly two times: in the forward direction (from i to j , where vertex i is closer to the root than vertex j) and in the opposite direction (from j to i).

We will build two lists for these edges. The first one will store the color of all edges in the forward direction, and the second one the color of all edges in the opposite direction. We will use 1 if the edge is colored, and 0 otherwise. Over these two lists we will build each a segment tree (for sum with a single modification), let's call them $T1$ and $T2$.

Let us answer a query of the form (i, j) , where i is the ancestor of j . We need to determine how many edges are painted on the path between i and j . Let's find i and j in the Euler tour for the first time, let it be the positions p and q (this can be done in $O(1)$ if we calculate these positions in advance during preprocessing). Then the **answer** to the query is the sum $T1[p..q-1]$ minus the sum $T2[p..q-1]$.